

Avora — The Life Engine

Information Architecture v1

0. The governing constraint

Thirteen life areas is a list of *concerns*, not a blueprint for *thirteen sections*. If Avora surfaces as 13 modules, it fails before day two — that's a dashboard with extra steps, and dashboards get abandoned.

The rule: one graph, many lenses.

There is exactly one underlying system of record — a graph of linked items. "Health," "Finance," "Pets," etc. are not databases. They're *filtered views* over the same graph. This isn't a simplification for its own sake — it's what makes cross-life intelligence possible at all. You cannot notice that a Reading goal, a Travel plan, and a Finance constraint are in tension with each other if they live in 13 separate silos. You can if they're nodes in one graph.

Everything below is designed against that constraint.

1. The Data Model

1.1 Universal item schema

Every single thing in Avora — a task, a note, a goal, a vet appointment, a book, a bill — is an instance of one underlying shape:

```
Item {
  id
  type: capture | task | event | note | goal |
project |
      person | place | resource | routine
  title
  body (optional)
  status: inbox | active | waiting | done |
archived
  created_at / updated_at
  source: manual | voice | email | calendar_sync
|
      brain_generated | import
  areas: [tag] — zero or more life-area tags
(not ownership, just lenses)
  links: [item_id] — typed edges to other items
  when: date/datetime/range (optional)
  effort: quick | short | deep (optional, user
or AI-estimated)
  energy_required: low | medium | high
(optional)
}
```

Everything inherits this. A "Health module" doesn't need its own task engine — a task tagged `area: health` linked to a goal tagged `area: health` is the health module, viewed through a filter.

1.2 Typed edges (what "relationships between modules" actually means)

The intelligence lives in the edges, not the nodes:

Edge type	Example
supports	Task → Goal
contains	Project → Task
blocks	Task → Task
concerns	Note → Person / Place / Project
funds / costs	Finance transaction → Goal / Project / Travel
attends	Event → Person
owns	Person → Pet
informs	Note → Decision

This is what lets the Brain say something like: *"Your savings goal and this trip you're planning are pulling in opposite directions"* — because Finance and Travel are edges on the same graph, not two apps that don't talk.

1.3 The 13 "areas" are lenses, not modules

Health, Finance, Reading, Meal Planning, Home, Pets, Relationships, Travel, Work, plus the structural four (Calendar, Tasks, Projects, Goals, Notes) — these are **tags +**

a thin, purpose-built view, not separate feature sets. Each lens gets:

- A filtered slice of the graph (items tagged with that area)
- 0–2 area-specific automations (see §5)
- Optionally, one area-specific input type where generic fields genuinely don't fit (e.g., Finance needs an amount + currency; Meal Planning needs ingredients)

That's it. No area gets its own navigation paradigm, its own "home screen," or its own onboarding flow. Consistency of interaction is what keeps cognitive load flat as the system scales — that's the actual promise of "reduces overwhelm," made structural instead of aspirational.

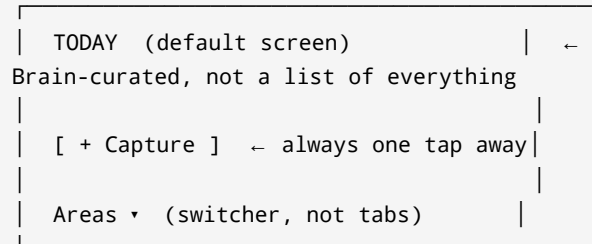
2. Core Modules (functional layers, not nav items)

Five functions, not thirteen sections:

- 1. Capture** — the only front door. Zero-friction, zero-categorization at time of entry.
- 2. Clarify** — the Brain triages raw capture into typed, linked items.
- 3. Organize** — the underlying graph: Goals → Projects → Tasks, Calendar, Notes.
- 4. Areas** — lenses over the graph for the 13 domains.
- 5. Reflect** — review surfaces (daily/weekly), powered by the Brain, not manually assembled.

Navigation exposes 3 things a person actually needs in a day: **Today**, **Capture**, **Areas** (as a drawer/switcher, not a tab bar). Reflect is a rhythm, not a place you "go" — it comes to you.

3. Navigation



- **Today:** the Brain's answer to "what now" — a short, ranked surface (see §4), not a dumping ground of every open task.
- **Capture:** global, always reachable (voice, text, or forward-an-email). This is the single most important interaction in the product — it must never ask a question at the moment of capture. Categorization happens later, invisibly.
- **Areas:** a switcher/drawer, not a tab bar with 13 icons. You go to "Finance" when you're thinking about finance. You don't see 13 icons every time you open the app — that visual load is the overwhelm.
- **Reflect:** surfaces itself (end of day, end of week) rather than living as a destination you have to remember to

visit.

No settings-heavy per-module navigation. No separate apps-within-the-app feeling.

4. The Brain — where it actually earns its keep

The Brain's job description in your brief is right; here's what it means structurally:

Brain function	Mechanism
Connects across areas	Reads the typed-edge graph, not per-module logic
Prioritizes	Ranks Today's surface by: due proximity + energy match (time of day, stated capacity) + goal-alignment weight + staleness (things quietly going stale get gently resurfaced, not nagged)
Suggests next best action	One suggestion at a time, not a list of options — a list is a new decision to make
Reduces overwhelm	Actively hides more than it shows. Success metric isn't items surfaced — it's items <i>correctly withheld</i>

Brain function	Mechanism
Learns behavior	Notices patterns (e.g., tasks tagged "quick" that sit for 3 weeks aren't actually quick – reclassify silently)
Never creates unnecessary work	The Brain is not allowed to generate a task that requires the user to manage the task. If it can just <i>do</i> the thing (reschedule, merge duplicates, close a stale loop), it does.

The unglamorous but critical rule: the Brain's default action on ambiguous capture is *not* "ask the user a clarifying question." It's "make a reasonable guess, tag it low-confidence, and let it get corrected in passing" – because every clarifying question is a cognitive-load tax at exactly the moment (capture) where the product promised zero friction.

5. Automatic vs. requires input

Automatic	Requires input
Categorizing captures into type/area	The initial capture itself (thought, task, idea)
Linking related items (same person, same project)	Setting a goal (its meaning is inherently subjective)

Automatic	Requires input
Recurring task/routine generation	Confirming a Brain suggestion when stakes are real (money, commitments to people)
Calendar ↔ task time-blocking	Reflection prompts' actual answers
Financial transaction categorization (if linked account)	One-time subjective inputs: energy level, mood, "how did that go"
Staleness detection & gentle resurfacing	Deciding what "done" means for fuzzy goals
Cross-area conflict detection (budget vs. trip, calendar vs. energy)	Anything involving another person's feelings or consent

General principle: **automate the bookkeeping, never automate the meaning-making**. Avora can reschedule a task. It should never decide a goal is abandoned without asking.

6. How information flows (a worked example)

1. You capture, voice note, walking the dog: *"need to figure out the vet situation before the trip"*

2. **Clarify:** Brain parses this into two linked draft items — a task (research/book vet) tagged area: pets , and a concerns edge to the existing Travel project for the trip — because it already knows about the trip on your calendar.
3. **Organize:** task gets a soft due date (Brain infers: before trip start date, minus buffer), effort estimated as "short."
4. **Today:** surfaces once, at a time you're usually doing errands/calls (learned pattern), not immediately and not repeatedly.
5. You do it or snooze it. If snoozed 3x, Brain doesn't nag — it asks once, gently, whether it's still relevant, then either lets it go stale visibly (not silently deleted — you'd lose trust) or re-scopes it.
6. **Reflect:** at week's end, this shows up only if it's still open — as one line in a short list, not a guilt report.

Nowhere in that flow did you open a "Pets module" or a "Travel module." You had a thought; the system did the filing.

7. What this deliberately leaves out (and why)

- **No per-area dashboards.** Thirteen dashboards is thirteen places to feel behind.
- **No streaks/gamification in the Engine.** That's the Town's job, and mixing it in here would make the Engine manipulative in exactly the way the brief says the Heart must *not* be.

- **No mandatory daily check-in.** Optional, never required — required rituals become another thing to fail at.
 - **Finance and Meal Planning stay shallow-native.** These are domains where dedicated best-in-class tools already exist (banks, meal apps). Avora's job is to *link and synthesize*, not to rebuild a budgeting app or a recipe database from scratch. Depth here is a trap — it's where "life OS" ambition quietly turns into feature bloat.
-

Open questions worth deciding next

1. How much should the Brain surface unprompted vs. wait to be asked — where's your actual tolerance line, given the "another tab to check" failure mode?
2. Does Reflect need to exist as a distinct rhythm, or could it just be Today with a wider lens on Sundays?
3. Which of the 13 areas, if any, genuinely need a native input type beyond title/note/date/amount?